

Générateur de programmes efficaces pour CoreWar

Nathan Giraud

Galéo Prioux

Gabriel Devick

19 avril 2026

Table des matières

1	Introduction	2
2	Structuration	2
3	Éléments techniques	2
3.1	Structures de données	2
3.1.1	Instructions	2
3.1.2	Warriors	3
3.1.3	Ram	3
3.1.4	Process	3
3.1.5	ProcessQueue	3
3.2	Algorithmes	4
3.2.1	L’algorithme génétique	4
3.2.2	Le recuit simulé	6
4	Architecture	8
4.1	Diagrammes de classes	8
4.2	Description des paquetages non standards utilisés	9
4.2.1	Apache Ant : Automatisation	9
4.2.2	JUnit : Tests unitaires	10
4.3	Architecture modèle-Vue-Contrôleur	10
4.3.1	Le modèle	10
4.3.2	La vue	10
4.3.3	Le contrôleur	10
4.3.4	Le pattern Observer	10
4.4	La chaîne de traitement	11
4.4.1	Phase 1 : Création	11
4.4.2	Phase 2 : Parsing	11
4.4.3	Phase 3 : Chargement en mémoire	11
4.4.4	Phase 4 : Cycle d’exécution	11
4.4.5	Phase 5 : Restitution	11
5	Expérimentations et usages	12
5.1	Interface graphique	12
5.2	L’algorithme génétique	12
5.3	Expérimentations du recuit simulé	14
6	Conclusion	16

1 Introduction

Le but de ce projet a été de créer un générateur de programmes efficaces pour le jeu Core-War. Ce jeu de programmation fait affronter deux programmes informatiques dans une machine virtuelle appelée MARS (Memory Array Redcode Simulator) pour en avoir le contrôle. Pour gagner, un programme doit forcer l'autre à se terminer en exécutant une instruction spécifique (DAT) avant un temps imparti, auquel cas il y a match nul. Pour répondre à ce défi, nous avons implémenté deux approches d'intelligences artificielles : un algorithme génétique et un algorithme de recuit simulé. Durant ce projet, nous avons rencontré quelques difficultés. La première a été d'implémenter une architecture solide (MVC) capable d'interpréter un langage RedCode et de gérer des processus dans une mémoire circulaire. Le point dur principal a été d'optimiser les performances du moteur pour les algorithmes de génération qui font combattre des milliers de guerriers. Dans un premier temps, on abordera la structure globale du projet et l'organisation de notre équipe. Ensuite, nous analyseront les éléments techniques tels que nos structures de données et nos algorithmes. Ensuite, nous détaillerons l'architecture logicielle grâce à des diagrammes et à la chaîne de traitement. Enfin, nous exposeront des résultats quantifiables de nos algorithmes.

2 Structuration

Pour répondre à ces problématiques, nous avons divisé notre projet en plusieurs étapes. La première a été le développement du langage RedCode. Nous avons ensuite dû créer une MARS, capable d'interpréter le langage RedCode pour faire combattre les guerriers. Finalement, nous avons dû mettre en place des algorithmes évolutionnaires pour générer des programmes performants.

En ce qui concerne les fonctionnalités implémentées, nous avons mis en place la version ICWS-88 (la première version) du jeu. Pour illustrer les combats, nous avons mis en place une interface graphique Swing. Cette interface permet d'écrire des programmes dans une zone de texte. Il y a également deux boutons de génération : un pour générer avec un algorithme génétique et un avec un algorithme de recuit simulé. Un autre bouton permet de charger les guerriers écrits ou générés dans la ram de la MARS et un autre permet de stopper le combat.

Pour ce qui est de l'organisation, voici comment nous nous sommes répartis la tâche. Nathan Giraud a implémenté la partie RedCode (qui comprend les instructions, les warriors et le parseur) ainsi que l'algorithme de recuit simulé. Galéo Prioux s'est occupé d'une partie de la MARS (la ram, la fifo et le loader) et de l'algorithme génétique. Gabriel Devick a mis en place toute la partie logique et arithmétique de la MARS (cu, alu, mars) et a aidé dans la mise en place du parseur et du loader. Il a également mis en place le design pattern *Observer* avec le package `mars.util` (ainsi que le fichier de config et la classe `Config`) ainsi que l'interface graphique (toute la classe `mars.gui`).

3 Éléments techniques

3.1 Structures de données

3.1.1 Instructions

La classe `Instruction` représente une instruction Redcode. Elle possède 5 attributs : un Op-Code (DAT, MOV, ADD, SUB, JMP, JMZ, JMN, CMP, SLT, SJN, SPL), deux modes d'adressages (DIRECT " ", IMMEDIAT #, INDIRECT @, ou PRECEDENT <), et deux valeurs entières correspondant aux opérandes A et B. Les modes adressages et OpCode sont définis comme des enum afin de garantir la validité des valeurs utilisées. Nous avons plusieurs constructeurs surchargés permettant de créer des instructions avec tous les paramètres ou en laissant

certaines valeurs par défaut. Elle possède aussi une méthode `copy()`, très utile pour l'algorithme génétique en particulier pour la fonction `crossover()` afin de ne pas modifier les parents lorsque l'on modifie l'enfant.

3.1.2 Warriors

La classe `Warrior` représente un programme `CoreWar` sous forme d'une `ArrayList<Instruction>`. On justifie ce choix pour l'accès direct aux index en $\Theta(1)$, très utile pour certaines fonctions qui manipulent les instructions par position. Elle possède deux constructeurs, l'un prend une liste d'instruction existante, l'autre crée un warrior vide auquel on ajoute des instructions via la méthode `addInstruction()`. Très important la méthode `toString()` permet de transformer le warrior en texte Redcode lisible, ce qui permet de le passer directement à la MARS pour les combats.

3.1.3 Ram

La RAM représente la mémoire circulaire de la machine virtuelle MARS. Pour simuler l'arène de combat avec précision, elle est structurée autour de deux `ArrayList` parallèles :

- **Une liste pour les instructions (`ArrayList<Instruction>`)** : Elle contient le code Redcode présent dans chaque cellule de l'arène. À l'initialisation de la machine, toutes les cases sont remplies avec l'instruction par défaut `DAT 0, 0`.
- **Une liste pour les propriétaires (`ArrayList<Int>`)** : Elle mémorise l'identifiant (ID) du dernier guerrier ayant écrit ou modifié la cellule correspondante. Par défaut, cette valeur est initialisée à 0, désignant une cellule neutre n'appartenant à aucun joueur.

Le rôle du système de propriétaires (*owners*) Ce système de doubles listes est une implémentation importante de notre projet. À chaque fois qu'un processus modifie une case mémoire (par exemple via une instruction `MOV`, `ADD` ou `SUB`), la méthode `write()` de la RAM met à jour l'instruction et l'identifiant de son nouveau propriétaire.

Cette fonctionnalité est utile uniquement pour faire le lien avec l'interface graphique. Elle permet au (`MarsPanel`) d'interroger la RAM pour associer une couleur spécifique à chaque case (par exemple, bleu pour le Guerrier 1 et rouge pour le Guerrier 2). L'utilisateur peut ainsi observer visuellement et en temps réel l'expansion d'un programme et la prise de contrôle de l'arène.

3.1.4 Process

La classe `Process` représente un fil d'exécution d'un warrior. Chaque processus possède deux attributs : un compteur (`pc`) qui indique l'adresse en RAM de la prochaine instruction à exécuter, et un identifiant de warrior (`warriorId`) qui permet de savoir à quel équipe appartient le processus.

3.1.5 ProcessQueue

La `ProcessQueue` permet d'ordonner les processus via une file FIFO implémentée avec une `LinkedList`. Les processus des deux warriors sont entrelacés dans la file, c'est-à-dire que les processus de chaque warrior arrivent les uns après les autres (`p1 w1 -> p1 w2 -> p2 w1 -> p2 w2` etc...). À chaque cycle le processus en tête est retiré (`dequeue`), son instruction est exécutée et si il survit il est réinséré en fin de liste (`enqueue`). Cela garantit une alternance équitable entre les deux warriors.

3.2 Algorithmes

3.2.1 L'algorithme génétique

L'algorithme génétique s'inspire de la théorie de l'évolution de Darwin.

Pour Darwin, le point de départ est une population aux « données » aléatoires, ici nous avons des Warriors. Comme le dit Darwin, ce qui permet de faire le tri et ainsi de créer l'évolution c'est l'environnement, ici nous utilisons les combats pour déterminer la valeur des « données ». Ces combats permettent de déterminer la qualité de notre combattant pour ensuite ne garder que les meilleurs. Cette population de supers guerriers, nous allons la coupler afin de mélanger leurs génomes, ou apporter des mutations afin de créer des variations et des potentiels guerriers aux « données » héritées des meilleurs. La population converge alors, génération après génération, vers des Warriors de plus en plus performants.

Le fonctionnement de notre algorithme se résume ainsi :

Algorithme 1 : AlgorithmeGénétique(taillePopulation, tailleCode, nbGénérations, tauxMutation)

```
population ← générer taillePopulation warriors aléatoires;
benchmarks ← {Imp(MOV 0, 1)};
pour chaque génération de 1 à nbGénérations faire
    pour chaque warrior  $w_i$  de la population faire
        Tirer 10 adversaires aléatoires;
        pour chaque adversaire faire
            résultat ← Combat( $w_i$ , adversaire) via la MARS;
            si victoire alors score( $w_i$ ) ← score( $w_i$ ) + 3;
            si nul alors score( $w_i$ ) ← score( $w_i$ ) + 1;
        pour chaque benchmark faire
            résultat ← Combat( $w_i$ , benchmark) via la MARS;
            si victoire alors score( $w_i$ ) ← score( $w_i$ ) + 6;
            si nul alors score( $w_i$ ) ← score( $w_i$ ) + 2;
    meilleur ← warrior avec le score le plus élevé;
    nouvellePopulation ← {copie(meilleur)};
    Ajouter 10% de warriors aléatoires dans nouvellePopulation;
    tant que nouvellePopulation n'est pas complète faire
        parent1 ← SélectionRoulette(population, scores);
        parent2 ← SélectionRoulette(population, scores);
        pointCoupure ← aléatoire entre 1 et tailleCode - 1;
        enfant ← parent1[1..pointCoupure] + parent2[pointCoupure+1..tailleCode];
        Muter(enfant, tauxMutation);
        Ajouter enfant dans nouvellePopulation;
    population ← nouvellePopulation;
retourner meilleur warrior de la population finale
```

Les Warriors sont représentés par une liste d'Instructions Redcode présentée précédemment. Cette liste, nous allons l'appeler « génome » du warrior. La population initiale est générée entièrement au hasard, c'est-à-dire que chaque warrior voit ses instructions aléatoirement construites via les méthodes `createRandomInstruction()` et `createRandomWarrior()`.

Afin de mesurer la qualité d'un warrior, nous utilisons donc le combat via la MARS. Celui-ci combat contre un échantillon de 10 adversaires tirés aléatoirement dans la population générée. Une victoire rapporte 3 points, le match nul 1 point, et la défaite 0. Nous avons également ajouté un benchmark fixe nommé Imp (`MOV 0, 1`), chaque victoire ou match nul contre ce warrior rapporte le double de points. Il sert de référence pour évaluer si notre warrior a une valeur stratégique ou non.

Nous avons choisi de créer une sorte de tournoi avec 10 warriors, plutôt que de faire jouer tous les warriors les uns contre les autres, pour éviter de devoir faire un nombre considérable de combats potentiellement inutiles. En effet, faire jouer des warriors nuls entre eux n'a pas forcément d'intérêt et l'attribution des points perdrait de sa pertinence.

Concernant le croisement entre la population de super warriors, il nous fallait une fonction pour sélectionner qui seraient les parents. Pour cela, on se base sur leur score : plus un individu a de points, plus il a de chances d'être choisi. Les individus qui ont un score de 0 peuvent quand même être choisis afin de préserver la diversité génétique.

Le croisement des parents utilise alors un point de coupure unique. Il est tiré aléatoirement entre la première et la dernière instruction, l'enfant hérite donc des instructions du parent 1 avant ce point et du parent 2 après. Ce tirage aléatoire du point de coupure crée davantage de combinaisons possibles et donc une meilleure diversité.

Après le croisement, les instructions de l'enfant ont une probabilité, correspondant au taux de mutation, d'être modifiées. Ces mutations se composent de 7 modifications : changement de l'opcode, changement du mode d'adressage (A ou B), modification d'une valeur sur l'opérande (delta de -5 à 5), échange de position entre deux instructions, et enfin changement intégral d'une instruction de manière aléatoire. Afin de ne pas détruire les bons patterns des warriors, on modère le taux de mutation à environ 10 %, sinon cela risquerait de réduire l'impact de la sélection sur leur consolidation.

À chaque génération, nous copions le meilleur warrior (meilleur score) dans la nouvelle génération, sans faire aucune modification. L'objectif est de garantir un niveau de performance croissant. Cependant, afin d'éviter que la population stagne et converge vers une mauvaise stratégie, nous remplaçons 10 % de la nouvelle population par des warriors générés aléatoirement. Nous répétons l'ensemble de ce processus sur N générations, et c'est le meilleur warrior qui est finalement retourné.

Concernant la complexité de l'algorithme nous sommes en $\Theta(PG)$ combats (nombre d'appelle de `mars.game()`), où P est la taille de la population et G le nombre de générations. Dans notre première version l'algorithme faisait combattre tous les warriors entre eux, à la place nous avons ajouter le "tournoi" de 10 warriors ce qui a réduit la complexité qui était alors en $\Theta(P^2 \times G)$. Afin de donner un ordre d'idée $P = 200$ et $G = 500$ on obtenait :

Sans tournoi : $P \times (P - 1) \times G \in \Theta(P^2 \times G) \Rightarrow 200 \times 199 \times 500 = 19\,900\,000$ combats

Avec tournoi : $P \times 11 \times G \in \Theta(P \times G) \Rightarrow 200 \times 11 \times 500 = 1\,100\,000$ combats

L'utilisation de ce tournoi divise alors le nombre de combats par ≈ 18 , permettant l'utilisation de l'algorithme sur des populations plus larges et un grand nombre de générations afin d'obtenir de meilleurs warriors.

3.2.2 Le recuit simulé

L'algorithme du recuit simulé s'inspire d'un processus utilisé en métallurgie : on chauffe un matériau à haute température, puis on le refroidit lentement pour stabiliser sa structure atomique et atteindre un état d'énergie minimale (le plus stable). En programmation, cette « énergie » correspond au score du Warrior, que nous cherchons à maximiser.

Le recuit simulé travaille sur un seul Warrior à la fois. Celui-ci est un guerrier initial généré aléatoirement via `createRandomWarrior()`. L'objectif est de transformer ce guerrier étape par étape pour le rendre de plus en plus performant.

L'algorithme repose principalement sur la variable *température* :

- **Au début** (température élevée) : l'algorithme est très souple. Il accepte de garder un Warrior même si ses performances diminuent légèrement. Cela permet de découvrir des Warriors potentiellement plus performants et d'éviter de rester bloqué trop tôt sur une solution moyenne (un optimum local).
- **À la fin** (température basse) : l'algorithme devient de plus en plus sélectif. Il n'accepte presque plus que les modifications qui améliorent le score du Warrior.

À chaque itération, la température diminue selon un taux de refroidissement `coolingRate`, jusqu'à atteindre une température minimale `minTemperature`, qui marque la fin de l'optimisation.

Pour mesurer la qualité du Warrior, la méthode `evaluate()` utilise un système de combat contre trois adversaires aux comportements distincts :

- **Le Bomber** : un adversaire qui bombarde la mémoire (Victoire : 30 pts, Nul : 7 pts).
- **L'Imp** : un adversaire simple qui se copie sur lui-même (Victoire : 20 pts, Nul : 5 pts).
- **Le Dwarf** : un guerrier plus complexe et robuste (Victoire : 50 pts, Nul : 10 pts).

Cette méthode garantit que le Warrior final sera capable de résister aux stratégies classiques du CoreWar, avec une pondération plus forte pour les adversaires difficiles comme le Dwarf.

Pour faire évoluer notre Warrior, nous générons un « voisin » via la méthode `findNeighbour()`. Ce voisin est une copie du guerrier actuel ayant subi une mutation aléatoire parmi cinq types :

- L'échange de deux instructions (Swap).
- Le changement de l'OpCode.
- La modification d'une valeur (delta de -5 à 5).
- Le changement du mode d'adressage.
- Le remplacement d'une instruction par une nouvelle.

C'est dans la méthode `acceptable()` que réside la logique centrale de l'algorithme. Si le voisin est meilleur, on le choisit comme nouveau guerrier actuel. S'il est moins bon, on peut quand même l'accepter selon une probabilité calculée par la formule :

$$P = e^{-\Delta/T}$$

où Δ est la différence de score et T la température. Plus la température est haute, plus on a de chances d'accepter un voisin moins bon, ce qui permet d'explorer davantage l'espace de recherche.

En fin de compte, l'algorithme conserve toujours une copie du meilleur Warrior rencontré durant tout le processus (`best`), et c'est celui-ci qui est retourné à la fin du programme.

Algorithme 2 : RecuitSimulé(température, tempMin, coolingRate)

```
current ← warrior aléatoire;
best ← copie(current);
scoreCourant ← Évaluer(current);
scoreMeilleur ← scoreCourant;
tant que température > tempMin faire
    voisin ← TrouverVoisin(current);
    scoreVoisin ← Évaluer(voisin);
    si scoreVoisin > scoreCourant alors
        current ← voisin;
        scoreCourant ← scoreVoisin;
    sinon
         $\Delta \leftarrow \text{scoreCourant} - \text{scoreVoisin};$ 
         $P \leftarrow e^{-\Delta/\text{température}};$ 
        si aléatoire() < P alors
            current ← voisin;
            scoreCourant ← scoreVoisin;
    si scoreCourant > scoreMeilleur alors
        best ← copie(current);
        scoreMeilleur ← scoreCourant;
    température ← température × (1 − coolingRate);
retourner best
```

4 Architecture

4.1 Diagrammes de classes

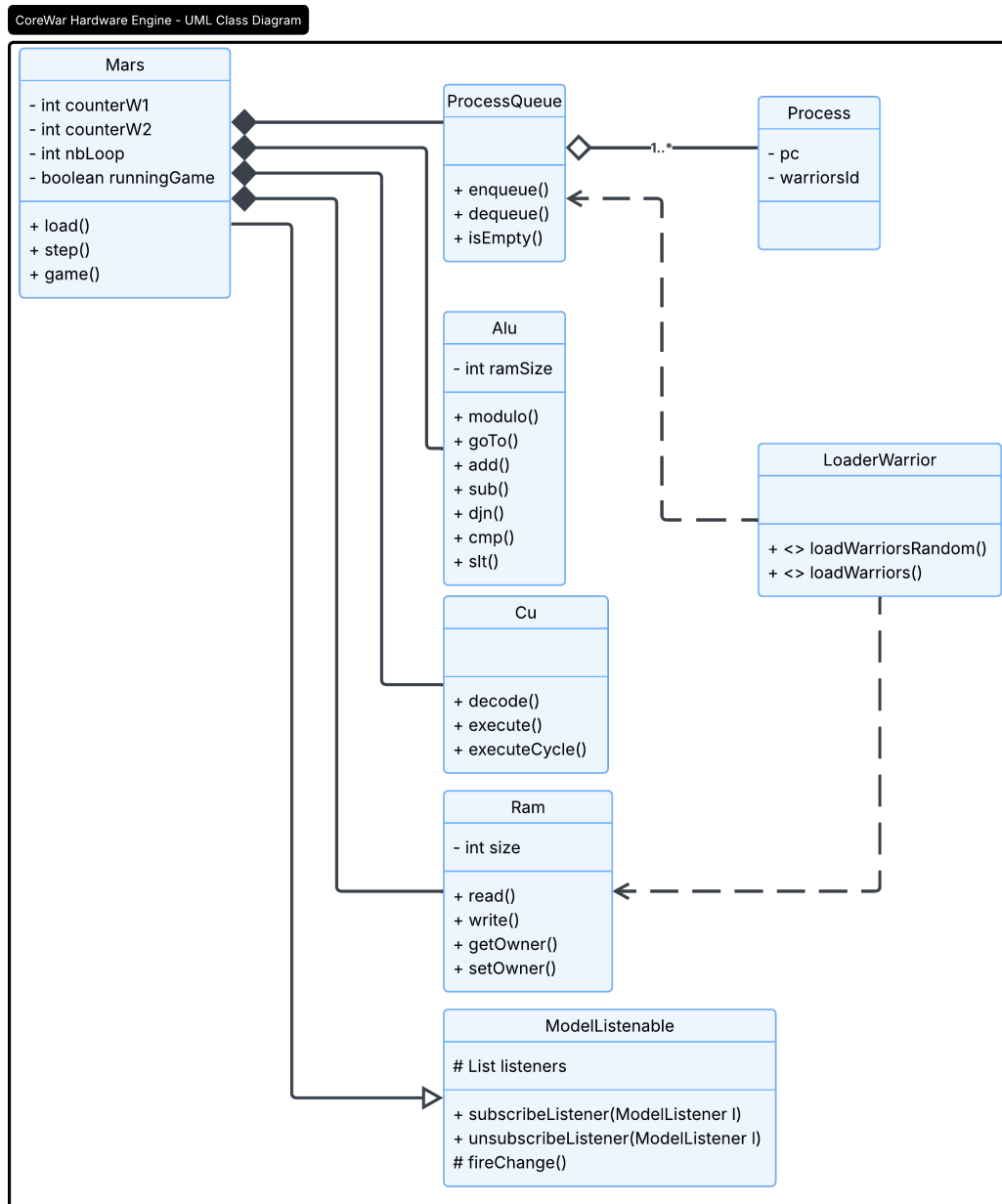


FIGURE 1 – Diagramme de classes – Package MARS

Les classes de ce package sont détaillées en section 3.1.

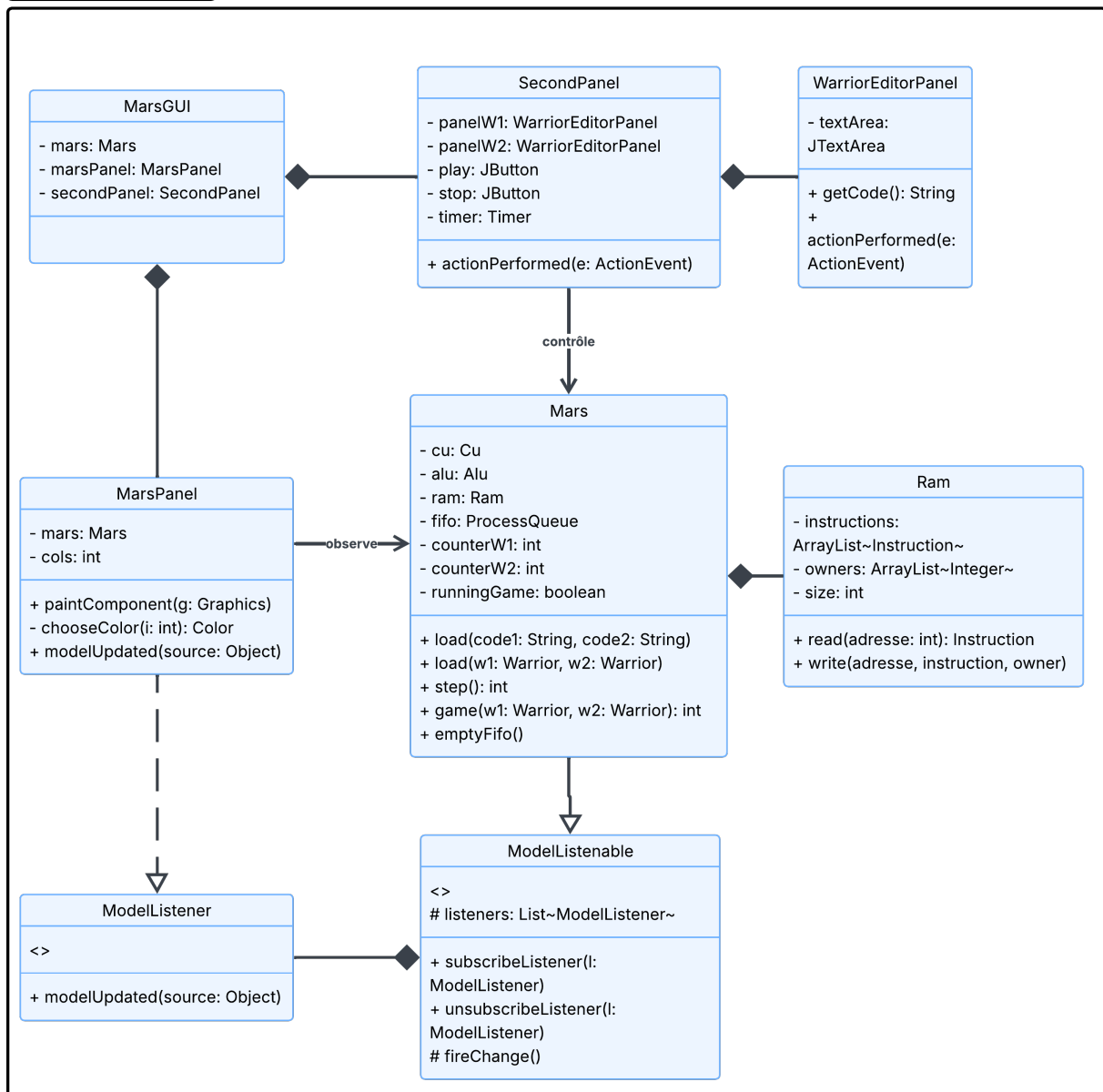


FIGURE 2 – Diagramme de classes – GUI

L'interface graphique suit le pattern MVC, détaillé en section 4.3.

4.2 Description des paquetages non standards utilisés

Bien que la plupart du projet ait été développée en utilisant uniquement les bibliothèques standards de Java, certaines parties ont nécessitées l'utilisation de deux outils externes : Apache Ant et JUnit.

4.2.1 Apache Ant : Automatisation

Ant est un logiciel de construction automatisée d'application, utilisant le XML. Pour ce projet, son utilisation s'est avérée utile pour recompilier et exécuter automatiquement, en évitant les lignes de commande. Grâce à un fichier `build.xml` à la racine du projet, nous avons pu définir des "cibles" (targets) pour gérer la compilation automatique :

- **Compilation** (`ant compile`) : Compilation centralisée de l'ensemble des sources avec gestion des dépendances.
- **Exécution** (`ant run`) : Lancement direct de l'interface graphique de la machine virtuelle.
- **Documentation** (`ant javadoc`) : Génération automatique de la documentation technique (Javadoc) pour l'ensemble de nos classes et méthodes.
- **Test** (`ant test`) : Lancement automatique des tests unitaires.

L'utilisation d'Ant garantit la simplicité de la compilation et de l'exécution pour tous les utilisateurs.

4.2.2 JUnit : Tests unitaires

JUnit est le framework de référence pour les tests unitaires. L'intégration de tests unitaires avec JUnit nous a permis de valider le comportement de nos composants de la MARS. Cela a particulièrement été utile pour :

- Vérifier la justesse mathématique de l'ALU et le calcul modulaire, garantissant que les adresses bouclent bien sur la mémoire circulaire.
- Valider la robustesse du parseur (`InstructionFactory.java`).

4.3 Architecture modèle-Vue-Contrôleur

Afin de séparer clairement la logique et l'interface graphique, nous avons choisis de mettre en place une architecture **Modèle-Vue-Contrôleur**. Cela facilite la mise à jour du code et permet de modifier l'interface graphique sans toucher au moteur.

4.3.1 Le modèle

Le modèle représente le coeur de la logique et les données du jeu. Dans ce projet, il est principalement représenté par la classe `Mars`. Ce modèle n'a aucune connaissance de l'interface graphique. Il se contente de gérer le jeu et notifier les observateurs lors des changements.

4.3.2 La vue

La vue est responsable de représenter visuellement le modèle. C'est le rôle de la classe `MarsPanel` en dessinant la grille représentant la mémoire. Elle interroge le modèle pour obtenir l'état de la `Ram` et les propriétaires de chaque case.

4.3.3 Le contrôleur

Le contrôleur fait le lien entre l'utilisateur et le modèle. Dans notre interface, les classes `SecondPanel` et `WarriorEditorPanel` remplissent cette tâche. Elles captent les actions de l'utilisateur (clics sur les boutons) et appellent les méthodes correspondantes.

4.3.4 Le pattern Observer

Pour que la vue puisse se mettre à jour sans que le modèle ne soit au courant, nous avons implémenté le pattern **Observer** :

- Le modèle hérite de `ModelListenable`
- La vue implémente l'interface `ModelListener` et s'abonne au modèle pour être notifié.
- A chaque changement dans le modèle, il appelle la méthode `fireChange()`, ce qui déclenche la méthode `repaint()` de la vue.

4.4 La chaîne de traitement

Pour comprendre l'architecture globale de notre projet, il est essentiel de comprendre la chaîne de traitement, le cycle de vie d'un programme, de sa création à son exécution finale.

4.4.1 Phase 1 : Création

Le début de la chaîne de traitement dépend du mode d'utilisation. Un programme peut être :

- **Saisi manuellement** par l'utilisateur via une zone de texte présente sur l'interface graphique (`WarriorEditPanel`).
- **Généré automatiquement** par l'un de nos deux algorithmes (soit génétique soit de recuit simulé).

4.4.2 Phase 2 : Parsing

Une fois le programme créé sous forme de caractères, il est transmis à l'`InstructionFactory`. Cette classe agit comme un compilateur :

1. Il nettoie le code (supprime les espaces superflus).
2. Il décode les chaînes pour créer des objets `Instruction`.
3. Il assemble les instructions dans une `ArrayList` pour créer une objet `Warrior`.

4.4.3 Phase 3 : Chargement en mémoire

Les objets `Warrior` sont donnés au `LoaderWarrior`. Cette classe a pour but de charger les instructions dans la `Ram`. Il place le premier guerrier au début de la mémoire et le deuxième à la moitié. De plus, il initialise le premier pointeur (le `Process`) de chaque guerrier et le place dans la `ProcessQueue`.

4.4.4 Phase 4 : Cycle d'exécution

La classe `Mars` orchestre les tours de boucle. A chaque cycle :

1. La machine retire le premier `Process` de la file.
2. La `Cu` lit l'instruction pointée dans la `Ram`.
3. L'`Alu` calcule les adresses (en appliquant le modulo de la mémoire circulaire).
4. La `Cu` exécute l'action (modification de la mémoire, saut...).
5. Si le processus survit (n'a pas exécuté un `DAT`), son pointeur est incrémenté et remplacé à la fin de la `ProcessQueue`.

4.4.5 Phase 5 : Restitution

La boucle s'arrête lorsqu'il ne reste plus de processus en vie pour l'un des deux guerriers, ou lorsque la limite de temps est atteinte. La chaîne de traitement se termine donc avec le retour du résultat :

- **En mode interface graphique** : La vue (`MarsPanel`) est rafraichie à chaque tour de boucle via le design pattern *Observer*, et une boîte de dialogue annonce le vainqueur à la fin.
- **En mode entraînement (IA)** : La `Mars` retourne soit 0 pour un match nul, 1 si le premier guerrier gagne et 2 si c'est le deuxième.

5 Expérimentations et usages

5.1 Interface graphique

L'image ci-dessous montre notre interface graphique. La partie supérieure de l'écran affiche l'état de la mémoire de la Mars en temps réel. Grâce au système de double liste expliqué en section 3.1.3, chaque case est colorée en fonction de l'identifiant de son propriétaire. La partie inférieure permet de saisir un programme manuellement, ou d'en générer un en choisissant un de nos deux algorithmes ainsi que de lancer ou stopper le combat.

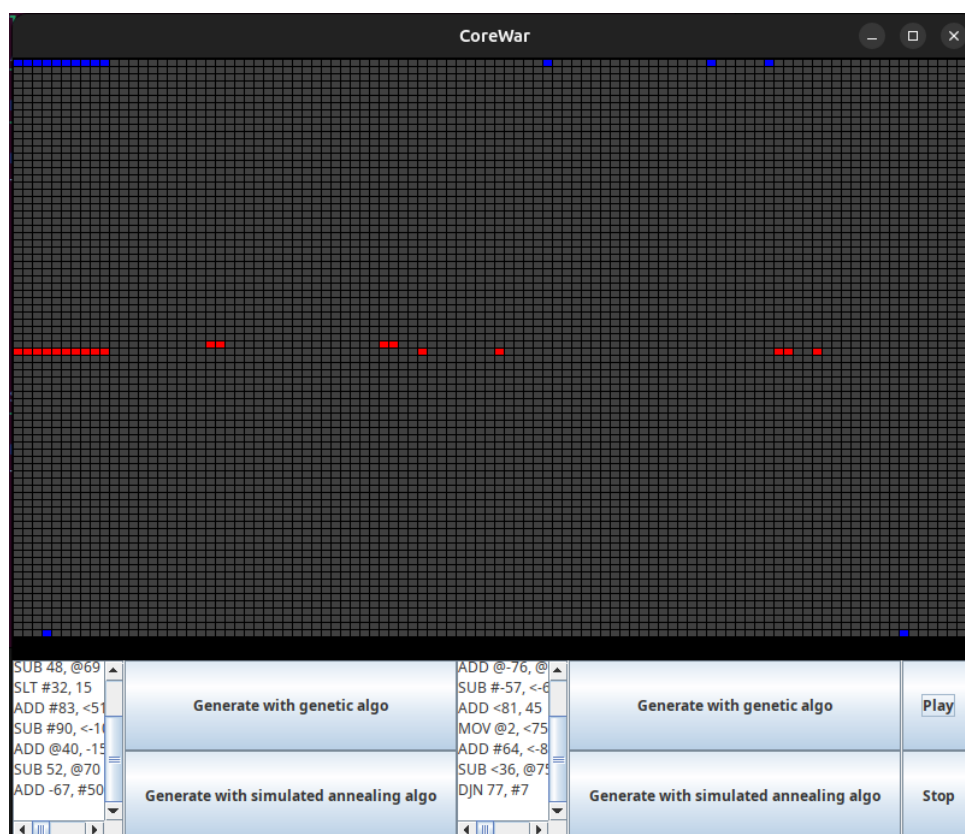


FIGURE 3 – Exécution d'un combat dans l'arène MARS via l'interface graphique

5.2 L'algorithme génétique

Les expérimentations suivantes portent exclusivement sur l'algorithme génétique. Le recuit simulé fait l'objet d'une analyse séparée. Chaque run a été évalué sur 20 combats contre le Dwarf afin de mesurer le winrate du meilleur warrior généré.

TABLE 1 – Résultats des expérimentations de l'algorithme génétique

Run	Population	Générations	CodeSize	Mutation	vs Dwarf
1	50	150	10	0.10	8 – 12
2	200	500	15	0.06	13 – 7
3	200	500	15	0.50	0 – 20
4	200	1000	15	0.08	16 – 4
5	50	50	15	0.10	0 – 20

Run 1 (8-12 vs Dwarf)	Run 2 (13-7 vs Dwarf)
MOV 7950, <4279 SPL 7999, <7488 SPL 7999, #3112 SPL 7997, <36 SPL 7999, #7950 SUB -23, 70 JMN #-61, @7984 DJN <71, @125 SUB @-75, <99 JMN @-72, @34	SPL 0, @7014 SPL -1, 7811 MOV <111, 7852 MOV <112, <4012 DAT <9, @90 DJN #7917, 2230 JMN -74, @3718 CMP #87, <7989 SUB <922, #3260 DAT #88, <5902 SLT <80, @5007 DAT 67, -33 SLT 25, 95 JMZ <3, @7052 ADD #7970, <7995

FIGURE 4 – Warriors générés – Runs 1 et 2

Run 3 (0-20 vs Dwarf)	Run 4 (16-4 vs Dwarf)
SLT 54, #7667 ADD <78, #7910 MOV <7989, @7854 JMN <90, 82 JMP 1, <7922 JMN -48, <7986 SUB @63, 7967 CMP #50, <7981 JMZ <-55, -75 MOV #-12, <44 JMN <7906, 7664 SUB -33, <-9 SUB -32, <-19 SPL #31, 37 SPL @-31, 74	SPL 0, <7821 MOV @7695, <4008 JMP 7998, 7284 MOV @7790, @6799 CMP <-49, @6257 JMP <7872, #3107 MOV 0, <6824 MOV @0, <7694 SPL 66, 7873 JMP <27, @7992 SPL @-16, -11 MOV 7948, <3989 CMP #27, @7693 JMZ @119, 7864 DAT #58, @7938

FIGURE 5 – Warriors générés – Runs 3 et 4

Run 5 (0-20 vs Dwarf)

```

MOV @12 , @93
SLT @7959 , #7871
SUB #62 , #4589
ADD @46 , <5453
ADD #7940 , #501
ADD @46 , @1902
MOV #88 , @2040
SLT <87 , @2
ADD @106 , 2086
MOV 27 , <7894
SLT @7926 , @2165
SUB <7945 , @7885
DJN 7997 , 5954
JMN @82 , 7987
SUB #62 , @2496

```

FIGURE 6 – Warrior généré – Run 5

Nous pouvons distinguer un pattern sur les warriors générés, les valeurs d’opérandes proches de 8000. En effet la mémoire de la MARS fait 8000 cases : donc une adresse de 7995 en mode direct revient à cibler 5 cases en arrière. On s’aperçoit alors que l’algorithme à découvert comment exploiter la mémoire circulaire pour adresser des cases proches.

Les expérimentations montrent que deux facteurs influencent fortement la qualité des warriors générés. Le nombre de générations est déterminant : avec seulement 50 générations (Run 5) on s’aperçoit que le warrior n’a pas les armes pour tenir tête au DWARF tandis qu’avec 1000 générations (Run 4) il atteint un winrate de 16-4. Le taux de mutation est lui aussi un point critique : à 50% (Run 4), l’algorithme produit un warrior aussi faible qu’avec trop peu de générations, cela s’explique par la potentielle destruction des bons patterns avant de les avoir consolidés. Le meilleur compromis selon ces tests se situe autour de 6 à 10% de mutation avec un grand nombre de générations.

5.3 Expérimentations du recuit simulé

Les expérimentations suivantes portent sur le recuit simulé. Pour chaque configuration, nous avons lancé plusieurs exécutions afin d’observer la progression du score entre le Warrior initial (généré aléatoirement) et le Warrior optimisé. Les paramètres de base sont : `temperature` = 8000.0 et `minTemperature` = 0.1. Le paramètre variable est le taux de refroidissement `coolingRate`.

TABLE 2 – Résultats du recuit simulé selon le taux de refroidissement

Run	coolingRate	Score initial	Score final
1	0.001	−45	55
2	0.001	−45	55
3	0.001	−15	55
4	0.0001	−45	55
5	0.0001	−45	55
6	0.00001	−45	55
7	0.00001	−45	55

Warrior initial (score : -45)	Warrior optimisé (score : 55)
SLT @11, 76	SUB <62, <-94
DAT #55, @77	JMP @5, @43
JMN @-78, -95	JMN -66, #-78
CMP @68, -30	MOV <-80, @92
JMP 60, <-59	JMN -12, <13
JMP @74, <46	JMN @-34, -38
SPL @-98, <-77	SUB <-78, -6
JMP <-44, #-5	SUB #-20, <47
DAT @95, #58	SPL 95, <-69
JMN #-40, @-88	ADD <68, #-3
SLT #-18, <-28	ADD @42, #-96
JMN #-96, 8	JMZ 100, <-55
MOV @-6, @40	DAT @81, <65
ADD #60, -4	DJN <-33, #-45
CMP #62, <-48	JMZ #-86, -22

FIGURE 7 – Exemple d’optimisation par recuit simulé (coolingRate = 0.001)

On observe que le recuit simulé converge systématiquement vers un score final de 55, quel que soit le score initial ou le taux de refroidissement. Un `coolingRate` plus faible (0.00001) permet un refroidissement plus lent et donc davantage d’itérations, mais le score final atteint reste identique. Cela suggère qu’un `coolingRate` de 0.001 est suffisant pour atteindre un bon résultat, tout en étant significativement plus rapide.

6 Conclusion

Ce projet avait pour objectif de développer un simulateur de CoreWar complet, de l'interprétation du langage Redcode à la générations de warriors performants. Nous avons commencé par créer une machine virtuelle MARS (Memory Array Redcode Simulator) capable de faire combattre deux programmes dans une mémoire circulaire, puis nous l'avons accompagnée d'une interface graphique permettant de visualiser les combats en temps réel grâce au pattern MVC et Observer.

Afin de répondre à la problématique de génération de programmes efficaces, nous avons décidé de suivre deux approches différentes avec deux algorithmes évolutionnaires différents. L'algorithme génétique, basé sur le Darwinisme, fait évoluer une population de warriors par la sélection, le croisement et la mutation. Il s'est révélé capable de produire des guerriers battant le Dwarf avec un très bon winrate dans les meilleures configurations. Le recuit simulé, qui optimise un seul warrior par mutations successives guidées par la température, converge de manière fiable vers un score stable de 55 points contre les benchmarks classiques du CoreWar.

Les expérimentations ont mis en évidence l'importance du calibrage des paramètres : un taux de mutation trop élevé ou un nombre de générations insuffisant produisent des warriors incapables de rivaliser avec les stratégies classiques. Nous avons également observé que nos algorithmes découvrent d'eux-mêmes des propriétés de la mémoire circulaire, comme l'utilisation de valeurs proches de 8000 pour adresser des cases voisines en arrière.

Plusieurs améliorations sont envisageables. Du côté de l'algorithme génétique, l'ajout de benchmarks supplémentaires dans l'évaluation (Dwarf, Stone) diversifierait la sélection et forcerait des pattern différents. La parallélisation des combats réduirait significativement le temps d'exécution. Enfin, s'il est possible d'envisager une co-évolutions, où deux populations évoluent l'une contre l'autre pourrait certainement créer des warriors encore plus performants et robustes.